

The POSIX port

**μOS++ III SMP on the development machine: one host thread
per CPU**

Port: micro-os-plus-iii-posix-arch

The POSIX port

How `micro-os-plus-iii-posix-arch` runs μ OS++ III SMP on the development machine, what "a CPU" means when there is no silicon, and the kernel defect it found on its first day.

This is the fourth architecture project, beside `-aarch32`, `-aarch64` and `-cortexm`. It is the only one that is not a migration: the other three carried code that had already been tested on hardware, while this one is new work, written against decision **D12** of the unification design — `posix-arch` must be SMP, modelled as one host thread per CPU.

1. What it is

```
micro-os-plus-iii-posix-arch/
├─ CMakeLists.txt
├─ include/
│  ├─ cmsis-plus/rtos/port/os-c-decls.h      the thread context (ucontext)
│  ├─ cmsis-plus/rtos/port/os-decls.h        the C++ port contract
│  ├─ cmsis-plus/rtos/port/os-inlines.h      criticals, CPU id, atomics
│  ├─ host_cpu.hpp                          the CPU model
│  ├─ exception_handler.hpp
│  └─ hw_result.hpp                        the verdict, as an exit status
├─ src/
│  ├─ rtos/os-core.cpp                      the port's half of the scheduler
│  ├─ host_cpu.cpp                          host threads as CPUs: bring-up, tick, IPI
│  ├─ exception_handler.cpp                SIGSEGV/SIGBUS/SIGFPE/SIGILL, reported
│  ├─ free-store.cpp                      the application heap
│  ├─ board-contract.cpp                  what a board must define, as #error
│  └─ diag/trace-posix.cpp
└─ test/
   ├─ CMakeLists.txt  run.sh
   ├─ boards/native/  board.cmake, include/, src/, run.sh
   └─ native/         that board's test applications
```

Roughly 2,300 lines, port and board together. One board, `native`, and ten test applications.

board	"silicon"	CPUs	tests
<code>native</code>	the host kernel	4 (<code>-DNCPU=</code>)	10

Nine of the ten are the SMP object tests carried unchanged from the ARM boards. The tenth, `mutex-stress`, is upstream's own and is the port's `OS_NCPU=1` leg — see §7.

2. The model: a host thread IS a CPU

`OS_NCPU` host threads are created at startup and never destroyed. Each one runs the scheduler and is, for every purpose the kernel can observe, a core. `µOS++` threads remain `ucontext` contexts switched **within** a CPU, which is upstream's machinery, preserved deliberately.

concern	ARM port	here
CPU id	<code>MPIDR_EL1</code>	<code>thread_local unsigned _this_cpu</code>
interrupt mask	<code>DAIF</code> / <code>CPSR.I</code>	<code>pthread_sigmask</code> on this thread
per-CPU tick	generic timer PPI	<code>timer_create</code> + <code>SIGEV_THREAD_ID</code>
IPI	GIC SGI / mailbox	<code>pthread_kill(SIGRTMIN+1)</code>
kernel lock	<code>ldxr</code> / <code>stxr</code> spinlock	<code>__atomic_exchange_n</code> spinlock
in-handler flag	<code>_in_isr[OS_NCPU]</code>	<code>_in_isr[OS_NCPU]</code> — identical
context switch	assembly save/restore	<code>swapcontext</code>
idle "WFI"	<code>dsb sy; wfi</code>	<code>sigsuspend</code>
secondary release	spin table / mailbox	<code>pthread_create</code>

The port deliberately does **not** link `micro-os-plus::port-smp-decls`. That target's own README names this port as the example that must keep its own declarations: the shared copy is written for ports whose state types are integers and whose thread context is a bare stack pointer, and neither is true here. The declarations the kernel actually reads — `lock_state[]`, `_smp_klock`, `_smp_tlock`, `_port_ctx_pending[]`, `_in_isr[]` — are reproduced with the same names, types and volatility, so the kernel cannot tell the two apart.

Preemption is real

Upstream's `NOTES.md` records the limitation this port removes: *"the scheduler runs in cooperative mode only; thread pre-emption might be possible, but was considered not worth the effort."*

Here every CPU has its own periodic timer and the context switch happens **inside the signal handler**, not after `sigreturn`. Two consequences are written into `host_cpu.cpp` and are not oversights:

- **No `SA_ONSTACK` on the tick.** The handler runs on the `µOS++` thread's own stack, so the whole signal frame is part of the context `swapcontext()` saves and therefore migrates with the thread. An alternate stack belongs to the *host* thread and would be left behind. This is why `OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES` is 32 KiB here rather than the ARM ports' 2 KiB.
- **`SA_ONSTACK` on the fault handler, and only there.** A fault handler never migrates — it reports and the process ends — and it must work when the stack it would otherwise use is the thing that broke.

The three defects D12 names

1. `sigprocmask` → `pthread_sigmask`. In a multithreaded process the behaviour of `sigprocmask` is unspecified. The replacement is also a conceptual win: a per-thread signal mask *is* per-CPU interrupt masking.
2. `setitimer(ITIMER_REAL)` → `timer_create` + `SIGEV_THREAD_ID`. `ITIMER_REAL` delivers `SIGALRM` to an arbitrary thread of the process, so with N CPUs the preemption of CPU 2 could be charged to CPU 0. A per-thread timer is what a per-core timer actually is. The signal is `SIGRTMIN`, not `SIGALRM`, because real-time signals queue rather than coalesce. Both are *functions*, not constants: glibc's `SIGRTMIN` expands to a call to `__libc_current_sigrtmin()`.
3. **Native TLS and migration.** Decided before the scheduler was allowed to migrate, not after: **migration is allowed, native TLS is banned.** No port or application state may live in `errno` or a `thread_local` across a switch point. The one `thread_local` the port keeps is `_this_cpu`, and it is correct precisely because it describes the *host thread*, which is the CPU — it is re-read after every switch and never cached across one.

This was verified rather than assumed: the built image contains no `__tls_get_addr`, and `port_cpu_id()` compiles to a single `mov %fs:-0x4,%eax; ret` — local-exec, no pointer held in a register.

One consequence of the same rule: the application free store is `μOS++`'s own `first_fit_top`, **not** glibc `malloc`. A `μOS++` thread can be preempted inside an allocation and resumed on a different host thread, and glibc's arena lock would then be released by a thread that never took it.

3. The deferred publish

This is the one genuinely hard part, and it is a kernel requirement the design spec does not mention.

The SMP picker in `micro-os-plus-iii-smp/src/rtos/os-core.cpp` skips any ready thread whose `context_.port_.stack_ptr` is null:

```
if (th != nullptr && is_thread_allowed_on_cpu (th, cpu)
    && (th == old_thread || th->context_.port_.stack_ptr != nullptr))
```

Null means *not safe to claim*: the thread's context is still live in another CPU's registers. On the ARM ports the assembly restore path publishes the value only after SP has left the outgoing stack.

There is no assembly here to publish from, and by the time `swapcontext()` returns this CPU is already running the *incoming* thread. So the port does two things:

- `stack_ptr` is the **first** member of `os_port_thread_context_t`, ahead of the `ucontext`. To the kernel it is a flag and the only question ever asked of it is whether it is null, so any stable non-null value serves; the port uses the address of the context's own `ucontext`.

- The publish is left for **whoever arrives next on this CPU**. `switch_stacks()` leaves the address and value in that CPU's slot (`host_cpu::defer_publish`), and the first thing any resumed context does — after `swapcontext()`, and in `host_cpu`'s trampoline for a context that has never run — is `host_cpu::publish_pending()`.

This is AArch64's `_smp_pub_addr`/`_smp_pub_val` pair: same hazard, same answer, different machine. No kernel change was needed for it.

Three further rules fall out of it, all of them paid for in debugging:

- **A fresh context starts with this CPU's interrupts masked.** `getcontext()` snapshots the caller's mask, and a thread is very often created inside a critical section, so inheriting it would mask the new thread for ever; clearing it is wrong the other way, because a tick landing in the trampoline *before* `publish_pending()` would overwrite the pending slot and strand the outgoing thread for good. So a new context begins the way a resumed one does: masked. The trampoline publishes, then unmask.
- **The whole of `switch_stacks()` runs with this CPU masked.** On ARM that is free — the function runs inside the IRQ path. Here `reschedule()` reaches it from thread mode, so without the mask a tick can re-enter a half-performed switch. The mask is saved into the outgoing context and restored from the incoming one, so it follows the thread across CPUs, which is what interrupt state should do.
- **The kernel lock is released owner/depth first, lock word last.** The same order, for the same reason, that the AArch64 port documents at length: storing the lock word first lets another CPU win the lock and install its own owner and depth, which the next two stores then wipe — and every CPU spins for ever. On the ARM ports that was the `smp_test4` hardware deadlock.

4. The defect this port found

`smp_test2` — four CPUs incrementing a counter under one μ OS++ mutex — faulted in 21 runs out of 25 at `OS_NCPU=4`, with `this == nullptr` inside `thread::priority_inherited(priority_t)`.

The fault is in the kernel, not the port:

```
// src/rtos/os-mutex.cpp, mutex::internal_try_lock_()
if ((boosted_prio_ > owner_->priority_inherited ()))
{
    scheduler::uncritical_section sucs; // releases the kernel lock
    owner_->priority_inherited (boosted_prio_);
}
```

`scheduler::uncritical_section` releases the kernel lock — it must, because `priority_inherited()` ends in a yield, which cannot run scheduler-locked. While it is open, the mutex's owner can complete its own `unlock()` on another CPU, and `internal_unlock_()` ends by setting `owner_` to `nullptr`. The compiler must re-read the member after the opaque call, so the second line dereferences a null pointer.

Every SMP port has this window. It is rare on hardware because `scheduler::unlock()` / `lock()` is a handful of instructions; here it is two `pthread_sigmask()` system calls, which widens it enough to make the fault the common case. **That is what a synthetic SMP host is for.**

The fix captures the owner under the lock, dereferences the captured pointer, and re-checks ownership inside the uncritical section — so a thread that released the mutex meanwhile is neither dereferenced nor left with an inherited priority no later `unlock()` would clear. With it, `smp_test2` passes 25 runs out of 25.

How it was found is worth recording, because three speculative fixes before it resolved nothing:

1. The port's fault reporter was taught to print the faulting `pc` from the signal's `ucontext`, the image's load bias (captured with `dladdr()` in `exception::init()`, which is not a signal-handler act) and a `backtrace()`.
2. `addr2line` on `pc - bias` named `thread::priority_inherited(unsigned char)`; the return address one frame up named `mutex::internal_try_lock_`, called from `mutex::lock()`.

That reporting is still there. It cost nothing and it is the difference between "it crashed" and a line number.

5. The board

`test/boards/native/` is thin, and honestly so. On the BCM2837 the equivalent files own the spin table, the mailbox doorbell and the cache maintenance around them, because releasing a core is a silicon act; here the silicon is the host kernel, so the act belongs to the port and the board only names it.

What the board does provide is the same API the ARM boards do, so the carried tests compile **unchanged**:

header	what it is here
<code>uart.hpp</code>	<code>uart::uart1</code> over <code>write(2)</code> — async-signal-safe, unbuffered
<code>led.hpp</code>	<code>[LED on]</code> / <code>[LED off]</code> , printed only on change
<code>timer_arm.hpp</code>	<code>CLOCK_MONOTONIC</code> , presented at 1 GHz
<code>smp.hpp</code>	<code>smp::start_secondary_cores()</code> and <code>g_core_stage[]</code>

`src/heap.cpp` is the one piece of assembly in the whole port, and it is there for a reason that is not about the CPU: the tests compute their heap size as `__heap_end - __heap_start` on **labels**, so the two must be labels and not objects. A `char* const` would have made that subtraction the distance between two pointers rather than the size of the block.

The SD card is a file

Two of the carried tests reach a block device. `micro-os-plus-iii-devices` gained a third `sd::SdCard` back-end for them, `soc/native/sd_hostfile.cpp`, over `pread` / `pwrite` on an image file — which is

exactly what `sd.hpp`'s dispatch comment says a new back-end should be, so the BCM2837 and RK3506 back-ends are byte-for-byte unchanged and neither test needed an edit.

```
UOS_BOARD_DEVICES  micro-os-plus::devices-hostfile
```

`UOS_SD_IMAGE` names the file (default `disk.img`), `UOS_SD_SECTORS` its size (default 65536 sectors = 32 MiB). An existing image grows but never shrinks. `test_smpl/run-host.sh` gives each such test its own image under the log directory.

Note that the test build is **not** a `HW_BUILD`. That define means "use the board's real FAT32 boot partition"; the SD tests' other branch is flatfs over a plain image file, which is what a host-file back-end is.

6. Building and running

```
cd micro-os-plus-iii-posix-arch
cmake -S . -B test/build -G Ninja -DBOARD=native \
      -DCMAKE_BUILD_TYPE=Release
cmake --build test/build -j8

BOARD=native test/run.sh          # the whole suite
BOARD=native test/run.sh smp_test2 # one test, echoed to this terminal
```

`-DNCPU=<n>` changes how many host threads act as CPUs (default 4).

`test/run.sh` is a dispatcher and nothing else, the sibling of the ARM ports' `test/qemu.sh`: it hands off to `test/boards/<id>/run.sh`, so adding a board adds a directory. The board's runner in turn calls the shared `test_smpl/run-host.sh`, which is `run-qemu.sh` with the emulator taken out — same timeout table, same log directory, same PASS/SKIP/FAIL/TIMEOUT verdicts, same summary line, same exit status.

```
mutex-stress          PASS
smp-mat-test           PASS
smp-num-test           PASS
smp-pipeline-test      PASS
smp-pro-cons-test      PASS
smp_test0              PASS
smp_test1              PASS
smp_test2              PASS
smp_test3              PASS
smp_test4              PASS

host suite: 10 passed, 0 skipped, 0 failed
```

A test "on the host" is simply a process, so `hw_result::ok()` / `fail()` always end the run and the verdict reaches the shell as the exit code — unlike the silicon ports, where they compile to a semihosted `SYS_EXIT` under `SEMIHOST` and to nothing otherwise. They flush stdio first: most tests here print through `uart::uart1`, which is `write(2)`, but a test carried from upstream's suite prints with `printf()`, and to a pipe that is fully buffered.

7. `OS_NCPU=1`

The port is dual-branch on `OS_USE_SMP_SCHEDULER`, which `uos_add_app()` derives from `NCPU GREATER 1`. This is the same arrangement `cortexm` uses to run its three STM32 boards at `OS_NCPU=1` off the RP2350's SMP core.

The surface is small, because the kernel spells only one thing two ways: `scheduler::current_thread_` is an array indexed by CPU under the SMP scheduler and a single pointer without it. `host_cpu::current_thread(cpu)` is that difference, in one place; `os_idle_thread_core[]` is the only other SMP-only symbol the port names.

The nine SMP object tests **cannot** be built single-core by construction: they use `thread::cpu_affinity()`, which does not exist without the SMP scheduler. So the single-core leg is `mutex-stress`, carried from the kernel's own `tests/sources/mutex-stress` — upstream's test, not one invented for the purpose, and the self-contained one of the two candidates (`rtos-apis` pulls in FatFs and posix-io). It is wired through the per-test `board_test_ncpu` hook, so one build tree holds nine four-CPU tests and one single-CPU test.

It is also apt: ten threads hammering one mutex for ten seconds is the shape of the defect in §4.

```
[ 15s] t0:107 t1:116 t2:125 t3:112 t4:115 t5:115 t6:123 t7:121 t8:126 t9:117
      sum=1177, avg=118, sigma=5, delta in [-11,8] [-8%,7%]
Done.

RESULT: PASS
```

8. What is not carried, and what is not done

Not carried — three tests that need real hardware and would test the host rather than the kernel: `sd_test` (a seeded FAT32 boot partition), `smp-mat-sdcard-test` (the same card, for a 4 GiB working set) and `usb_test` (USB device mode).

Not done:

- `rtos-apis`, upstream's larger API suite, at `OS_NCPU=1`. It would need FatFs and posix-io wired to the host.

- Sanitizers. ASan and TSan both have opinions about `swapcontext` and about a signal handler that never returns to where it was raised; making the port usable under them is its own piece of work, and it is the one thing a synthetic host ought eventually to be good at.
- More than one board. A second would be a different *host* — macOS, where `SIGEV_THREAD_ID` does not exist and the tick has to be a `kqueue` timer or a dispatch source. The directory structure is already ready for it.

9. Where things are

port	<code>micro-os-plus-iii-posix-arch</code>
kernel	<code>micro-os-plus-iii-smp</code> (submodule)
devices	<code>micro-os-plus-iii-devices</code> (submodule)
the SD back-end	<code>micro-os-plus-iii-devices/soc/native/</code>
the shared runner	<code>micro-os-plus-iii-smp/test_smpl/run-host.sh</code>
the design decision	<code>docs/specs/2026-09-20-...-unification-design.md</code> §7.6, D12